

4_핵심소스코드_A축

핵심 소스코드 — A축 (개인 GPS 습관 인증)

Booster — AI 습관 형성 서비스 / A축 근거: backend/src/main/java/com/booster/** 실제 구현 코드 양식:
[서식2] 제작설계서 PPTX 슬라이드 27:28 (코드 + 설명)

| 핵심 소스코드 (1) — GPS 반경 판정(Haversine 알고리즘)

핵심 적용 기술. 두 위경도 좌표 사이의 지표면 거리(m)를 구해 등록 반경과 비교한다.

```
// shared/gps/GpsVerificationEvaluator.java
@Component
public class GpsVerificationEvaluator {

    private static final double EARTH_RADIUS_METERS = 6_371_000.0;

    public boolean isWithinRadius(double registeredLat, double registeredLng,
                                   int radiusMeters,
                                   double currentLat, double currentLng) {
        double distance = haversineDistance(registeredLat, registeredLng, currentLat, currentLng);
        return distance <= radiusMeters;
    }

    private double haversineDistance(double lat1, double lng1, double lat2, double lng2) {
        double dLat = Math.toRadians(lat2 - lat1);
        double dLng = Math.toRadians(lng2 - lng1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
            + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
            * Math.sin(dLng / 2) * Math.sin(dLng / 2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return EARTH_RADIUS_METERS * c;
    }
}
```

설명: 지구를 반지름 6,371km 구로 근사하여 두 좌표의 대원거리(great-circle distance)를 계산한다. 외부 지도 API 호출 없이 서버 내 연산만으로 ms 단위 판정이 가능해, 인증 요청이 몰려도 외부 의존성 병목이 없다.

| 핵심 소스코드 (2) — 일일 GPS 인증 처리(핵심 비즈니스 흐름)

위치 확인 → 중복 방지 → 반경 판정 → 스트릭/출석/코인 보상까지 하나의 트랜잭션으로 처리.

```
// personalcheckin/service/PersonalCheckInService.java
@Transactional
public CheckInResponse checkIn(Long userId, double currentLat, double currentLng) {
    LocalDate today = LocalDate.now(clock);

    PersonalLocation location = personalLocationRepository.findById(userId)
        .orElseThrow(() -> BusinessException.badRequest(
            "LOCATION_NOT_REGISTERED", "개인 GPS 위치를 먼저 등록하세요."));

    // 당일 중복 인증 방지 (이미 SUCCESS면 409)
    PersonalCheckIn existing = personalCheckInRepository
        .findByUserIdAndDate(userId, today).orElse(null);
    if (existing != null && existing.isSuccess()) {
        throw BusinessException.conflict("DUPLICATE_CHECK_IN", "오늘 이미 인증을 완료했습니다.");
    }

    // GPS 반경 판정 - 실패 시 레코드 생성하지 않음
    boolean within = gpsEvaluator.isWithinRadius(
        location.getLat(), location.getLng(), location.getRadiusMeters(),
        currentLat, currentLng);
    if (!within) {
        throw BusinessException.badRequest("GPS_OUT_OF_RANGE", "등록된 위치 반경을 벗어났습니다.");
    }

    OffsetDateTime now = OffsetDateTime.now(clock);
    if (existing != null) {
        existing.markSuccess(now); // 복귀 대기 레코드 보정
    } else {
        personalCheckInRepository.save(PersonalCheckIn.success(userId, today, now));
    }

    Streak streak = streakRepository.findById(userId).orElseThrow(...);
    streak.recordSuccess(today); // 스트릭 +1

    User user = userRepository.findById(userId).orElseThrow(...);
    user.increaseAttendance(); // 출석 +1

    boolean rewardGranted = false;
    if (isRewardEligible(streak.getCurrentStreak())) { // 마일스톤 도달 시
        coinService.grant(userId, rewardCoins, CoinTransactionReason.STREAK_REWARD, null);
        rewardGranted = true;
    }
    return new CheckInResponse(today, SUCCESS, now,
        streak.getCurrentStreak(), streak.getMaxStreak(), user.getCoinBalance(), rewardGranted);
}
```

설명: UNIQUE(user_id, check_in_date) 제약 + 애플리케이션 단 중복 검사로 하루 1회 인증을 보장한다. GPS 실패 시에는 레코드를 만들지 않아 데이터가 오염되지 않으며, 인증·스트릭·출석·코인 변경이 하나의 @Transactional 로 묶여 부분 반영을 방지한다.

| 핵심 소스코드 (3) — 코인 단일 진실 원천(Single Source of Truth)

모든 코인 변동을 한 메서드로만 처리해 잔액 = 거래내역 합계 불변식을 보장한다.

```
// coin/service/CoinService.java
@Transactional
public long charge(Long userId, long amount, CoinTransactionReason reason, Long referenceId) {
    if (amount < 0) throw new IllegalArgumentException("charge amount must be >= 0: " + amount);
    User user = lockUser(userId); // 비관적 락 (동시 차감 방지)
    long effective = Math.min(amount, user.getCoinBalance()); // 잔액까지만 차감(음수 방지)
    user.addCoins(-effective);
    coinTransactionRepository.save(
        CoinTransaction.of(userId, reason, -effective, user.getCoinBalance(), referenceId));
    return effective; // 실제 차감액 반환
}

private User lockUser(Long userId) {
    return userRepository.findByIdForUpdate(userId) // SELECT ... FOR UPDATE
        .orElseThrow(() -> BusinessException.notFound("USER_NOT_FOUND", "사용자를 찾을 수 없습니다."));
}
```

설명: 지급(grant)·차감(charge)이 항상 CoinTransaction 기록과 User.coinBalance 갱신을 함께 수행하므로 SUM(amount) == coinBalance 가 항상 성립한다. SELECT ... FOR UPDATE 비관적 락으로 동시 요청에도 잔액 정확성이 깨지지 않으며, 부하 테스트(BS-30)에서 동시성 오류 0건으로 검증되었다.

(참조) 데이터 모델 — A축 테이블

테이블	PK	주요 컬럼	관계/제약
users	id	email(unique), password_hash, nickname, coin_balance, total_attendance, is_active, joined_at	모든 A축 도메인의 기준
personal_locations	user_id	lat, lng, radius_meters, place_name	user_id 공유(1:1)
streaks	user_id	current_streak, max_streak, last_success_date	user_id 공유(1:1)
personal_check_ins	id	user_id, check_in_date, status, verified_at	UNIQUE(user_id, check_in_date)
recovery_missions	id	personal_check_in_id(unique), user_id, deadline_at, completed_at, status	check_in과 1:1
coin_transactions	id	user_id, type, amount(부호 있음), balance_after, reference_id	코인 변동 단일 이력

※ 연관관계는 JPA 객체 매핑이 아니라 **논리 FK(Long id)**로 설계 → 의도적으로 N+1 쿼리 0건 (BS-30 검증). ※
status enum: PersonalCheckIn = SUCCESS / RECOVERY_PENDING / FAILED , Recovery = PENDING /
COMPLETED / FAILED .